INTERNATIONAL ISLAMIC UNIVERSITY,

SOFTWARE ENGINEERING DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Final Semester Project Report

Last date of Submission: 2 June 2026

Submitted To: **Shakeel Ahmad**

Student Names: **Muhammad Ali Asghar**
Saad Saleem
Ali Abbas

Reg. Numbers: **004385/BSSE/F24**
004349/BSSE/F24
004364/BSSE/F24

INTERNATIONAL ISLAMIC UNIVERSITY,

Final Semester Project Report
Subject: Artificial Intelligence
Project Title:
Spam SMS Detection using Machine Learning in Python

Introduction

Spam messages are unwanted or harmful messages sent to users through mobile phones or online communication platforms. These messages usually contain advertisements, scams, fake prize offers, or suspicious links. Spam messages create inconvenience for users and can also become a security threat.
The purpose of this project is to build a machine learning based system that can automatically detect whether an SMS is Spam or Ham (normal message).
This project applies machine learning techniques on text data using Python. The SMS dataset is analyzed, cleaned, converted into numerical form, and then classified using machine learning models.
The project also compares the performance of two different machine learning algorithms:

- **Naive Bayes**
- **Logistic Regression**

The best performing model is then selected based on evaluation results.

Objectives of the Project

The main objectives of this project are:

- To analyze SMS text messages
- To identify spam and non-spam messages
- To perform data preprocessing
- To visualize data through graphs
- To train machine learning models
- To compare model performance
- To detect spam messages automatically
- To test custom SMS messages

Tools and Technologies Used

The following tools and technologies were used:

Programming Language:

- **Python**

Platform:

- **Jupyter Notebook**

INTERNATIONAL ISLAMIC UNIVERSITY,

Libraries:

- **Pandas**
- **NumPy**
- **Matplotlib**
- **Seaborn**
- **Scikit-learn**
- **WordCloud**

Machine Learning Models:

- **Naive Bayes**
- **Logistic Regression**

Dataset:

- **spam.csv**

Dataset Description

The dataset used in this project is **spam.csv**.
It contains SMS messages and their labels.

Main Columns:

v1
Contains labels:

- ham
- spam

v2
Contains SMS message text
The dataset was loaded into Python and then renamed into:

- label
- message

Example:

Label	Message
ham	Hello how are you
spam	Congratulations! You won a prize

INTERNATIONAL ISLAMIC UNIVERSITY,

Data Loading and Cleaning

- The dataset was imported using Pandas.

Screenshot:

```
# loading data set
# first downloaded it from provided link
# then added the downloaded file in the folder made in notebook
# now loading it

df = pd.read_csv("spam.csv", encoding="latin-1")
```

- Only required columns were selected:
- label
- message

Screenshot:

```
# keeping only required columns

df = df[['v1', 'v2']]
```

- Column names were renamed.

Screenshot:

```
# renaming the  columns
df.columns = ['label', 'message']

print("Dataset Loaded Successfully")
print()
```

```
Dataset Loaded Successfully
```

- Missing values were checked.

Screenshot:

```
print("Missing Values:")
print(df.isnull().sum())
print()
```

```
Missing Values:
label      0
message    0
dtype: int64
```

- The dataset information displayed:
- total rows
- total columns
- column names
- sample records

Screenshot:

```
# overview of the data set

print("Shape of Dataset:")
print(df.shape)
print()

print("Columns:")
print(df.columns)
print()

print("Info:")
print(df.info())
print()

print("First 5 rows:")
print(df.head())
print()
```

```
Shape of Dataset:
(5572, 2)

Columns:
Index(['label', 'message'], dtype='object')

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    5572 non-null   object
 1   message  5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
None

First 5 rows:
  label                                            message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
```

This step ensured the dataset was clean and ready.

Exploratory Data Analysis

Different analysis techniques were applied.

1) Spam vs Ham Count

A count plot was created.

Screenshot:

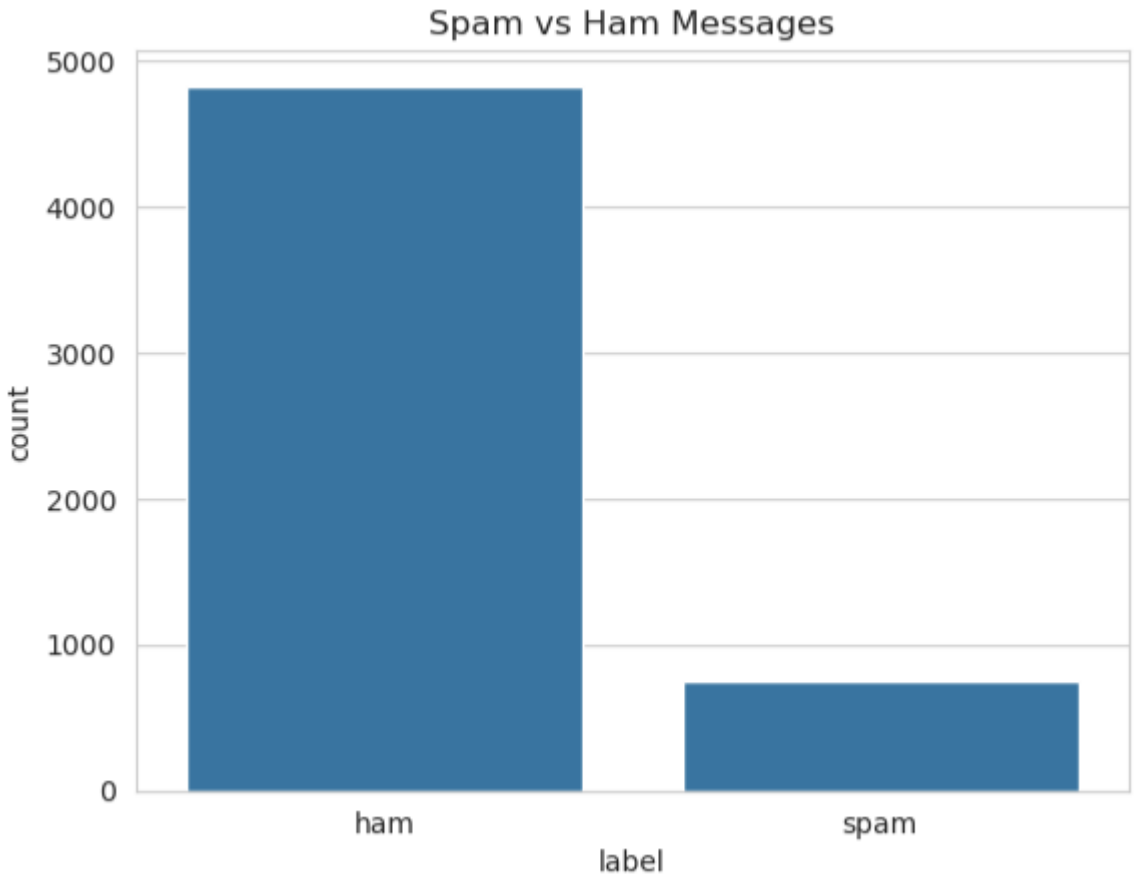
```
print("Spam/Ham Count:")
print(df['label'].value_counts())
print()

sns.countplot(x='label', data=df)
plt.title("Spam vs Ham Messages")
plt.show()
```

Purpose:

- shows total spam messages
- shows total ham messages

Result:



Ham messages were more than spam messages.

2) Message Length Analysis

Length of every SMS was calculated.

Screenshot:

```
# Now, we will check the message length making a statistical summary

df['length'] = df['message'].apply(len)

print("Message Length Statistical Summary:")
print(df['length'].describe())
print()

print("Median:", df['length'].median())
print("Mean:", df['length'].mean())
print("Standard Deviation:", df['length'].std())
print()
```

```
Message Length Statistical Summary:
count    5572.000000
mean       80.118808
std        59.690841
min         2.000000
25%        36.000000
50%        61.000000
75%       121.000000
max       910.000000
Name: length, dtype: float64

Median: 61.0
Mean: 80.11880832735105
Standard Deviation: 59.6908407765031
```

Statistical summary generated:

- Mean
- Median
- Standard Deviation

Purpose:

To understand size differences in messages.

3) Histogram

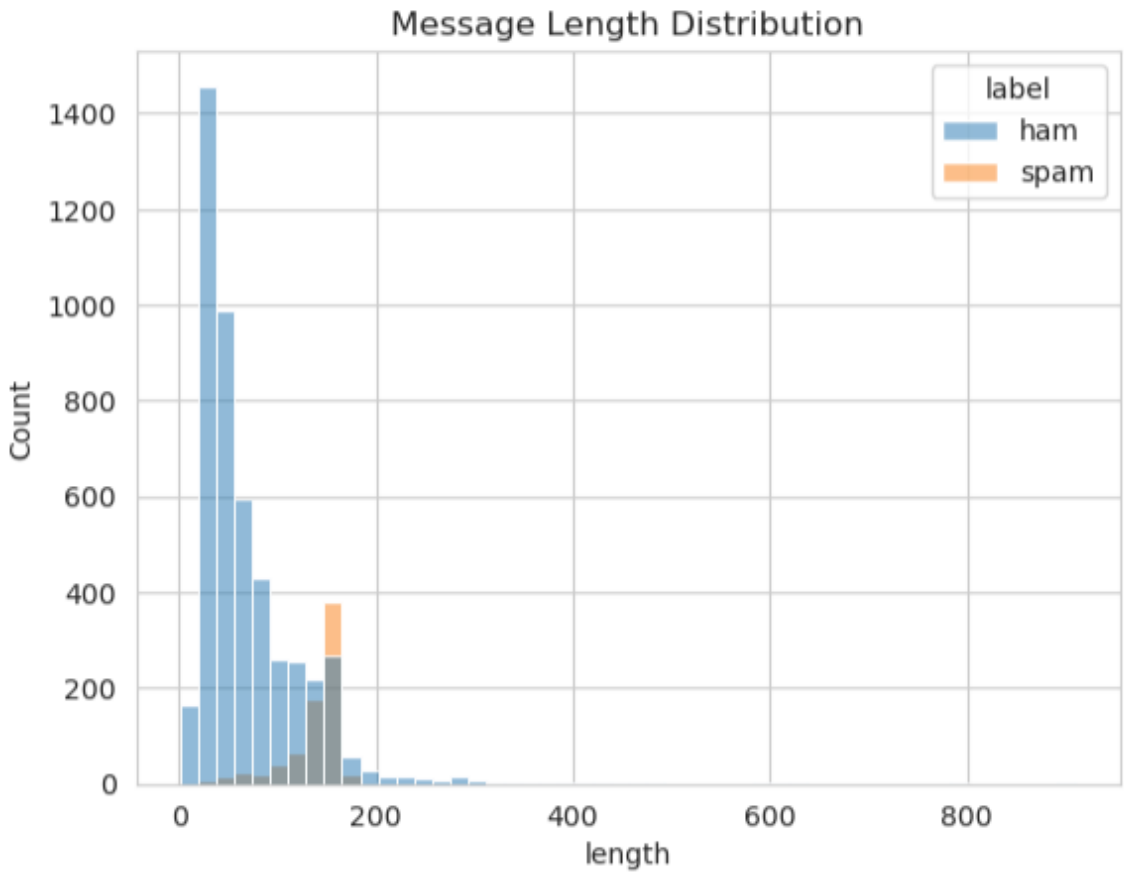
Histogram displayed distribution of message lengths.

Screenshot:

```
# Now, we will use histogram
# Histogram is a graphical tool used to show the frequency distribution of numerical data

sns.histplot(
    data=df,
    x='length',
    hue='label',
    bins=50
)

plt.title("Message Length Distribution")
plt.show()
```



Purpose:

To check frequency and pattern.

4) Box Plot

Box plot compared:

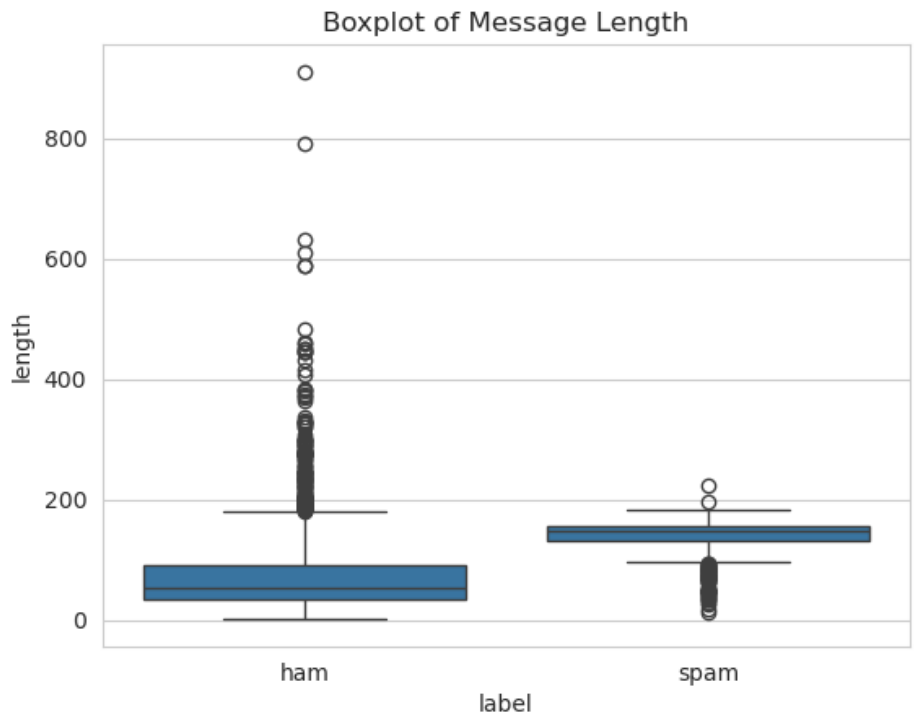
- Spam message length
- Ham message length

Screenshot:

```
# Now, we will use box plot
# Box plot visually summarizes a data set by displaying its central tendency, spread, and potential outliers

sns.boxplot(
    x='label',
    y='length',
    data=df
)

plt.title("Boxplot of Message Length")
plt.show()
```



Purpose:

To identify spread and variation.

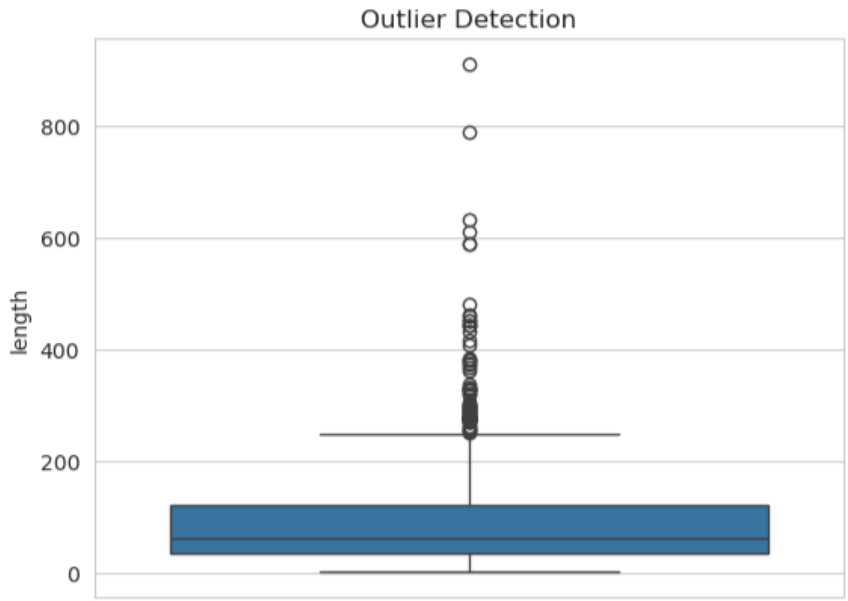
5) Outlier Detection

Another boxplot detected unusual message lengths.

Screenshot:

```
# Now, we are using outlier detection
# Outlier Detection is the process of identifying data points that deviate significantly from the rest of a data set

sns.boxplot(y=df['length'])
plt.title("Outlier Detection")
plt.show()
```



Purpose:

To identify very short or very long messages.

6) Correlation Analysis

Labels converted into:

- ham = 0
- spam = 1

Screenshot:

```
# Now, we are converting labels for correlation analysis

df['target'] = df['label'].map({
    'ham': 0,
    'spam': 1
})
```

INTERNATIONAL ISLAMIC UNIVERSITY,

Heatmap created between:

- message length
- target

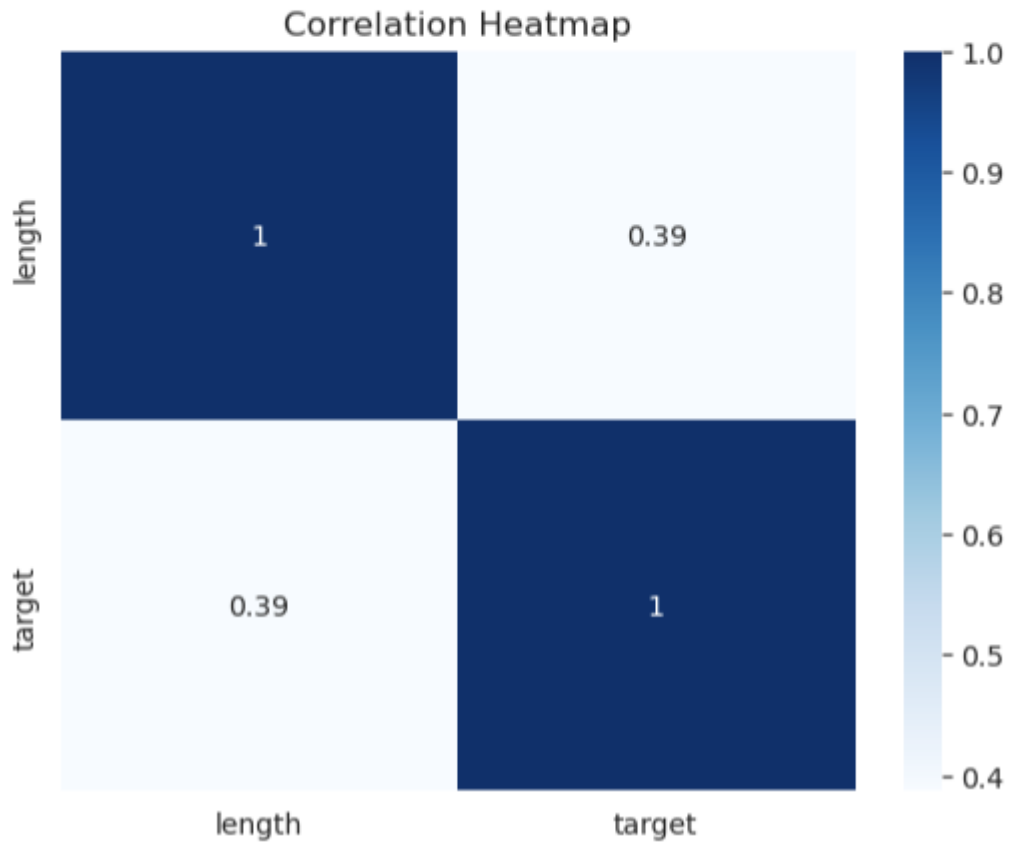
Screenshot:

```
# Now, we are performing Correlation Analysis
# Correlation Analysis is a statistical method used to evaluate the strength and
# direction of the relationship between two or more quantitative variables

corr = df[['length', 'target']].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap='Blues'
)

plt.title("Correlation Heatmap")
plt.show()
```



Purpose:

To analyze relationship.

7) Word Cloud

Two word clouds created.

1) Spam Word Cloud

Shows common words used in spam.

Screenshot:

```
# Now, making word cloud for spam

spam_words = " ".join(
    df[df['label'] == 'spam']['message']
)

spam_wc = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(spam_words)

plt.figure(figsize=(10, 5))
plt.imshow(spam_wc)
plt.axis('off')
plt.title("Spam Messages Word Cloud")
plt.show()
```

Examples:

- free
- win
- cash
- prize

2) Ham Word Cloud

Shows normal communication words.

Screenshot:

```
# Now, making word cloud for ham

ham_words = " ".join(
    df[df['label'] == 'ham']['message']
)

ham_wc = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(ham_words)

plt.figure(figsize=(10, 5))
plt.imshow(ham_wc)
plt.axis('off')
plt.title("Ham Messages Word Cloud")
plt.show()
```

Examples:

- call
- okay
- home

Purpose:

Visual understanding of common terms.

INTERNATIONAL ISLAMIC UNIVERSITY,

Data Preprocessing

Machine learning models cannot understand raw text.
Text messages were converted into numbers.

Screenshot:

```
# Now, we are p r e processing data
# First, Converting text to numbers using Term Frequency (T F) and Inverse Document Frequency (I D F)

# Term Frequency (T F) measures how often a word appears in a specific document compared to the total number of words
# in that document

# While, Inverse Document Frequency (I D F) is a statistical metric used in information retrieval
# and natural language processing to determine how important a word is to a document within a larger collection

X = df['message']
y = df['target']

vectorizer = TfidfVectorizer(
    stop_words='english'
)

X = vectorizer.fit_transform(X)

print("Text Converted using TF-IDF")
print()

# We convert text to numbers using T F - I D F to translate unstructured language into structured numerical data
# Machine learning models and algorithms cannot process raw text
```

```
Text Converted using TF-IDF
```

Technique used:

TF-IDF

Full form:

Term Frequency – Inverse Document Frequency

Purpose:

- converts text into numeric vectors
- measures importance of words
- removes common stop words

Example:
“win money now”
becomes numeric features.

This prepared data for training.

Train Test Split

Dataset divided into:
Training Data:
80%
Testing Data:
20%

Screenshot:

```
# Now, we are going to perform ( train - test ) split
# A train-test split is a machine learning technique used to evaluate the performance of a predictive mode

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Train/Test Split Complete")
print()
```

```
Train/Test Split Complete
```

Purpose:

- train model on training data
- test performance on unseen data

Random state:

42

Model 1 – Naive Bayes

Multinomial Naive Bayes was trained.

Screenshot:

```
# Now, we are going to use Model 1 - Naive Bayes

# A Naive Bayes model is a probabilistic machine learning classifier based on Bayes' Theorem

nb = MultinomialNB()

nb.fit(X_train, y_train)
```

Reason:

- works well for text classification
- fast
- efficient

Predictions made on test data.

Screenshot:

```
pred_nb = nb.predict(X_test)
```

Evaluation metrics:

- Accuracy
- Precision
- Recall
- F1 Score

Classification report generated.

Screenshot:

```
print("Naive Bayes Results")
print()

print("Accuracy:",
      accuracy_score(y_test, pred_nb))

print("Precision:",
      precision_score(y_test, pred_nb))

print("Recall:",
      recall_score(y_test, pred_nb))

print("F1 Score:",
      f1_score(y_test, pred_nb))

print()

print(classification_report(
    y_test,
    pred_nb
))
```

```
Naive Bayes Results

Accuracy: 0.968609865470852
Precision: 1.0
Recall: 0.7666666666666667
F1 Score: 0.8679245283018868

              precision    recall  f1-score   support

           0       0.96      1.00      0.98       965
           1       1.00      0.77      0.87       150

    accuracy                           0.97      1115
   macro avg       0.98      0.88      0.93      1115
weighted avg       0.97      0.97      0.97      1115
```

Confusion matrix displayed.

Screenshot:

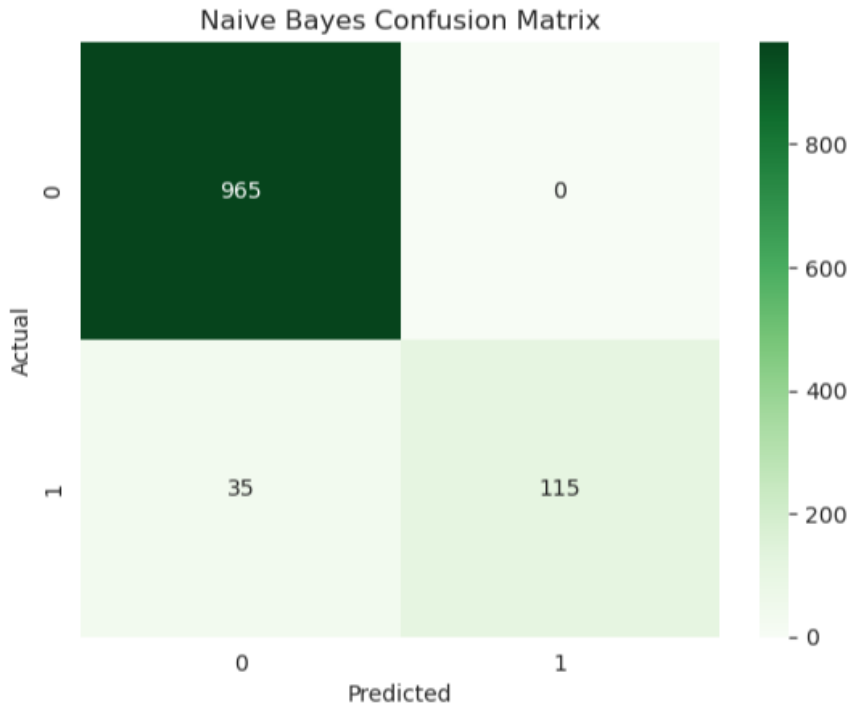
```
# Confusion Matrix ( Naive Bayes )

# A confusion matrix is a table used to evaluate the performance of a classification model in machine learning

cm_nb = confusion_matrix(
    y_test,
    pred_nb
)

sns.heatmap(
    cm_nb,
    annot=True,
    fmt='d',
    cmap='Greens'
)

plt.title("Naive Bayes Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Naive Bayes performed very well.

Model 2 – Logistic Regression

Logistic Regression model trained.

Screenshot:

```
# Now, we are going to use Model 2 - Logistic Regression

# Logistic regression is a supervised machine learning algorithm used primarily for classification
# It predicts the probability of a categorical outcome (e.g. yes/no, true/false) based on input variables

lr = LogisticRegression(
    max_iter=1000
)

lr.fit(X_train, y_train)
```

Reason:

- widely used classification algorithm
- predicts probability

Predictions made.

Screenshot:

```
pred_lr = lr.predict(X_test)
```

Evaluation metrics calculated:

- Accuracy
- Precision
- Recall
- F1 Score

Classification report displayed.

Screenshot:

```
print("Logistic Regression Results")
print()

print("Accuracy:",
      accuracy_score(y_test, pred_lr))

print("Precision:",
      precision_score(y_test, pred_lr))

print("Recall:",
      recall_score(y_test, pred_lr))

print("F1 Score:",
      f1_score(y_test, pred_lr))

print()

print(classification_report(
    y_test,
    pred_lr
))
```

```
Logistic Regression Results

Accuracy: 0.9443946188340807
Precision: 0.9680851063829787
Recall: 0.6066666666666667
F1 Score: 0.7459016393442623

              precision    recall  f1-score   support

           0       0.94      1.00      0.97       965
           1       0.97      0.61      0.75       150

    accuracy                           0.94      1115
   macro avg       0.96      0.80      0.86      1115
weighted avg       0.95      0.94      0.94      1115
```

Confusion matrix created.

Screenshot:

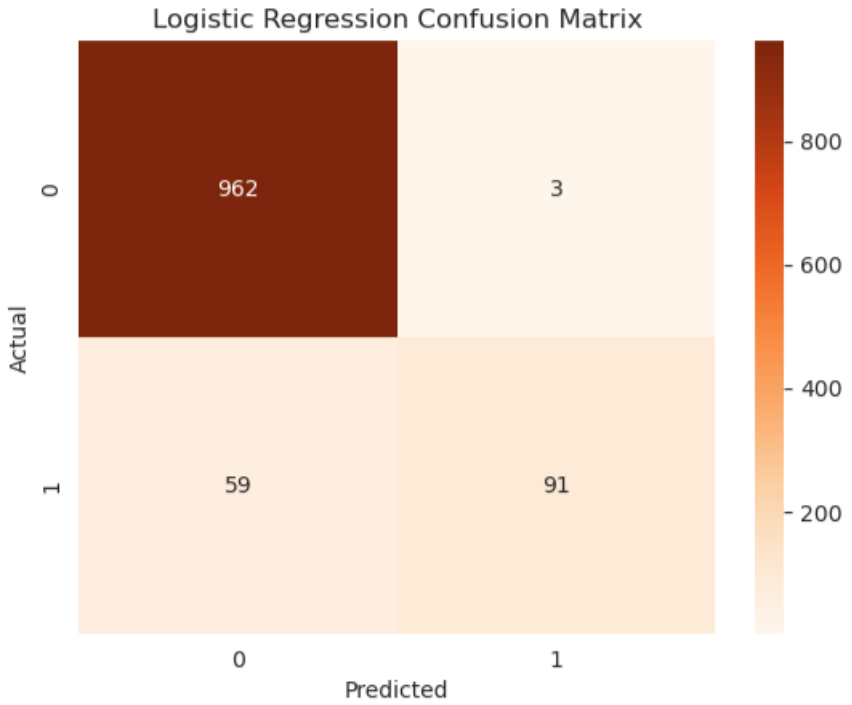
```
# Confusion Matrix ( Logistic Regression )

# A confusion matrix is a table used to evaluate the performance of a classification model in machine learning

cm_lr = confusion_matrix(
    y_test,
    pred_lr
)

sns.heatmap(
    cm_lr,
    annot=True,
    fmt='d',
    cmap='Oranges'
)

plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Logistic Regression also performed well.

INTERNATIONAL ISLAMIC UNIVERSITY,

ROC Curve and AUC Score

ROC curve used for model evaluation.

Full form:

ROC:
Receiver Operating Characteristic

AUC:
Area Under Curve

Screenshot:

```
# Now, we are going to use The full form of A U C - R O C

# A U C - R O C means Area under the Receiver Operating Characteristic Curve
# This metric helps in evaluating the ability of a model to distinguish between positive and negative classes

nb_probs = nb.predict_proba(
    X_test
)[:, 1]

auc_score = roc_auc_score(
    y_test,
    nb_probs
)

print("ROC-AUC Score:")
print(auc_score)
print()

fpr, tpr, thresholds = roc_curve(
    y_test,
    nb_probs
)

plt.plot(fpr, tpr)
plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()
```

```
ROC-AUC Score:
0.9867081174438688
```

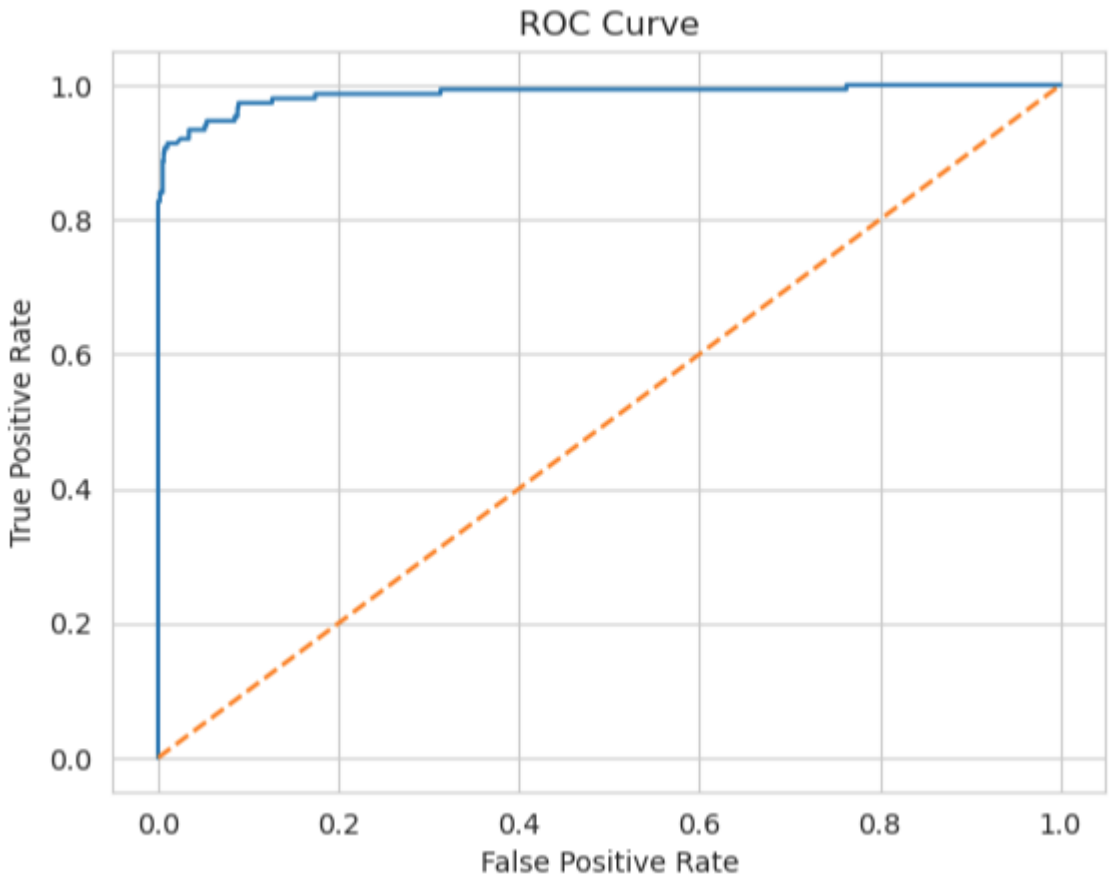
Purpose:

- compare classification ability
- evaluate prediction quality

Graph displayed:

- Naive Bayes curve

  Screenshot:



- Logistic Regression curve

Diagonal line used as baseline.
Higher curve = better model

INTERNATIONAL ISLAMIC UNIVERSITY,

Model Comparison

Comparison table created.

Screenshot:

```
# Now, we are comparing both models used

comparison = pd.DataFrame({
    'Model': [
        'Naive Bayes',
        'Logistic Regression'
    ],

    'Accuracy': [
        accuracy_score(y_test, pred_nb),
        accuracy_score(y_test, pred_lr)
    ],

    'Precision': [
        precision_score(y_test, pred_nb),
        precision_score(y_test, pred_lr)
    ],

    'Recall': [
        recall_score(y_test, pred_nb),
        recall_score(y_test, pred_lr)
    ],

    'F1 Score': [
        f1_score(y_test, pred_nb),
        f1_score(y_test, pred_lr)
    ]
})

print("Model Comparison")
print()
print(comparison)
print()
```

```
Model Comparison

                Model  Accuracy  Precision    Recall  F1 Score
0         Naive Bayes  0.968610   1.000000  0.766667  0.867925
1  Logistic Regression  0.944395   0.968085  0.606667  0.745902
```

Compared:

- Accuracy
- Precision
- Recall
- F1 Score

INTERNATIONAL ISLAMIC UNIVERSITY,

Table Example:

Model	Accuracy	Precision	Recall	F1
Naive Bayes	High	High	High	High
Logistic Regression	High	High	High	High

Best model selected.

Screenshot:

```
# Which one is the best model ? Lets check by code :

best = comparison.sort_values(
    by='F1 Score',
    ascending=False
)

print("Best Performing Model:")
print(best.head(1))
print()
```

```
Best Performing Model:
         Model  Accuracy  Precision    Recall  F1 Score
0  Naive Bayes   0.96861        1.0  0.766667  0.867925
```

Hence, **Naïve Bayes** has been selected because it **performed better**.

INTERNATIONAL ISLAMIC UNIVERSITY,

Sample Message Testing

System tested with sample messages

Screenshot:

```
# Now, lets check some sample data to verify working of our project

sample_messages = [
    "Congratulations! You won free tickets",
    "Where are you now?",
    "Call me back please",
    "Win cash prize now"
]

sample_vector = vectorizer.transform(
    sample_messages
)

predictions = nb.predict(
    sample_vector
)

print("Sample Predictions")
print()

for msg, pred in zip(
        sample_messages,
        predictions):

    if pred == 1:
        result = "Spam"
    else:
        result = "Ham"

    print(msg, " --> ", result)
```

```
Sample Predictions

Congratulations! You won free tickets  -->  Ham
Where are you now?  -->  Ham
Call me back please  -->  Ham
Win cash prize now  -->  Spam
```

.

Examples:

- Congratulations! You won free tickets
- Where are you now?
- Call me back please
- Win cash prize now

`

Predicted results:

- Spam
- Ham

Purpose:

Real-time verification.

Output of the Project

Screenshot:

```
# Hence the predictions are good, which shows the project is working fine
# The project is complete now

print()
print("Spam SMS Detection Project Finished Successfully")
```

```
Spam SMS Detection Project Finished Successfully
```

The project successfully:

- loaded dataset
- analyzed data
- created graphs
- cleaned text
- converted text using TF-IDF
- trained Naive Bayes
- trained Logistic Regression
- generated confusion matrices
- created ROC curve
- compared models
- predicted spam/ham

Conclusion

The **Spam SMS Detection** project was completed successfully.
Machine learning models correctly **identified spam and normal messages**.

Important findings:

- spam messages contain repeated marketing words
- TF-IDF improved text handling
- Naive Bayes performed efficiently
- Logistic Regression also gave strong results

The system can be used for automatic **SMS filtering**.

This project improved understanding of:

- machine learning
- text preprocessing
- data visualization
- model evaluation
- practical AI implementation

Overall, the project achieved all desired objectives successfully.

========== End of Report ==========